

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО»
ИНСТИТУТ КОМПЬЮТЕРНЫХ НАУК И КИБЕРБЕЗОПАСНОСТИ
ВЫСШАЯ ШКОЛА ПРОГРАММНОЙ ИНЖЕНЕРИИ

**Отчет о прохождении
стационарной ознакомительной, учебной практики на тему:
«Задача о рюкзаке с реализацией и сравнением полного
перебора, жадного алгоритма, динамического
программирования и алгоритма имитации отжига»**

Жуковского Егора Витальевича, гр. 5130903/40003

Направление подготовки: 09.03.03 Прикладная информатика

Место прохождения практики: СПбПУ, ИКНК, ВШ ПИ

(указывается наименование профильной организации или наименование структурного подразделения)

ФГАОУ ВО «СПбПУ», фактический адрес)

Сроки практики: с 10.06.2026 по 08.07.2026.

Руководитель практики от ФГАОУ ВО «СПбПУ»:

(Ф.И.О., уч. степень, должность)

Консультант практики от профильной организации:

Жаранова Анастасия Олеговна

(Ф.И.О., должность)

Оценка:

Руководитель практики
от ФГАОУ ВО «СПбПУ»

Консультант практики
от ФГАОУ ВО «СПбПУ»

Обучающийся

Жуковский Е.В.

Дата:

Введение

Задача о рюкзаке 0/1 является одной из главных проблем комбинаторной оптимизации, относящейся к классу NP-полных. Она формулируется следующим образом: имеется набор из n предметов, каждый из которых обладает весом w_i и стоимостью v_i , а также дан рюкзак с ограниченной вместимостью W . Требуется выбрать такое подмножество предметов, чтобы суммарная стоимость была максимальной, а суммарный вес не превышал вместимость. На практике задача используется в логистике, управлении запасами, распределении ресурсов, криптографии и других областях, где требуется исключить или включить элементы, в зависимости от ограничения.

Актуальность темы обусловлена необходимостью выбирать между точными и приближенными методами решения в зависимости от размерности входных данных и требуемой скорости вычисления. Точные алгоритмы (полный перебор, динамическое программирование) обеспечивают нахождение оптимального решения, однако их вычислительная сложность делает их невыгодными для больших n или больших значений вместимости. Приближенные методы (жадный алгоритм, метаэвристики) работают значительно быстрее, однако не гарантируют оптимальности. Сравнительный анализ различных подходов позволяет выбрать подходящий метод в зависимости от ситуации.

Цель работы – реализация, тестирование и сравнительный анализ четырех алгоритмов решения задачи о рюкзаке 0/1 (полный перебор, жадный алгоритм, динамическое программирование, имитация отжига), а также создание программы для генерации тестовых данных, автоматизированного бенчмаркинга, визуализации результатов и демонстрации через веб-интерфейс.

Для достижения поставленной цели были сформулированы и решены следующие задачи:

1. Разработать генераторы случайных тестовых наборов предметов и специальных “провокационных” примеров, на которых жадный алгоритм дает заведомо неоптимальное решение;
2. Реализовать четыре алгоритма: полный перебор (для малых размерностей), жадный алгоритм на основе сортировки по удельной стоимости, динамическое программирование с восстановлением выбранных предметов, имитацию отжига с представлением решения в виде битовой строки и штрафом за превышение веса;
3. Создать скрипт автоматизированного тестирования, обеспечивающий многократную генерацию данных, замер времени выполнения, сравнение полученной стоимости с эталонным решением и расчет точности в процентах с сохранением результатов в структурированный CSV-файл;
4. Реализовать модуль визуализации для построения графиков зависимости времени выполнения от размерности задачи, точности приближенных методов, а также числа итераций имитации отжига;

5. Разработать веб-интерфейс (стек React + FastAPI) с формой ввода параметров, возможностью запуска всех алгоритмов, выводом таблицы результатов и доступом к построенным графикам;
6. Провести вычислительный эксперимент на наборах данных различного объема (от 5 до 100 предметов) с многократной генерацией случайных наборов, зафиксировать и проанализировать полученные результаты;
7. Выполнить анализ эффективности алгоритмов, определить области их применимости и оформить итоговый отчет по практике.

Теоретическая часть

Формально задача о рюкзаке 0/1 определяется следующим образом. Дано множество предметов $I = \{1, 2, \dots, n\}$, каждому предмету i соответствуют вес $w_i > 0$ и стоимость $v_i > 0$. Требуется найти подмножество $S \subseteq I$, такое что $\sum_{i \in S} v_i$ – максимально, при ограничении $\sum_{i \in S} w_i \leq W$. Переменные принятия решений бинарны: $x_i = 1$, если предмет включен в рюкзак, иначе – $x_i = 0$. Задача NP-полна, т.е. для нее не известно полиномиального алгоритма точного решения.

В литературе предложен широкий выбор методов – от точных алгоритмов (полный перебор, ветви и границы, динамическое программирование) до приближённых и метаэвристических (жадные стратегии, генетические алгоритмы, имитация отжига и др.). В рамках данной работы рассматриваются четыре алгоритма, отличающиеся по подходу и вычислительной сложности:

Полный перебор (brute force) заключается в проверке всех 2^n возможных подмножеств, выборе подмножества с максимальной стоимостью. Этот алгоритм гарантирует точное решение, но имеет экспоненциальную временную сложность и $O(2^n)$ (практический предел – около 20–25 предметов).

Жадный алгоритм (greedy) основан на сортировке предметов по убыванию удельной стоимости v_i/w_i и последовательном добавлении предметов в рюкзак, пока позволяет вместимость. Вычислительная сложность определяется сортировкой и составляет $O(n \log n)$. Алгоритм работает очень быстро, но не обеспечивает оптимальности, есть риск ошибок.

Динамическое программирование (dynamic programming, DP) использует таблицу размером $(n + 1) \times (W + 1)$. где элемент $dp[i][j]$ – максимальная стоимость, достижимая при использовании первых i предметов и вместимости j . Рекуррентное соотношение:

$$dp[i][j] = \max(dp[i - 1][j], dp[i - 1][j - w_i] + v_i) \text{ при } j \geq w_i.$$

Алгоритм обеспечивает точное решение и имеет псевдополиномиальную сложность $O(nW)$. Он эффективен, когда вместимость W не слишком велика, но становится неприменимым при больших $W \approx 10^9$.

Имитация отжига (simulated annealing, SA) – метаэвристика, вдохновлённая физическим процессом отжига металлов. Решение получается в виде битовой строки длины n . Алгоритм начинает со случайного решения и итеративно генерирует соседние состояния (инверсия одного бита). Переход в состояние с лучшим значением целевой функции всегда принимается; переход в худшее состояние принимается с вероятностью, зависящей от текущей температуры. Температура снижается по заданному закону охлаждения. Для учета ограничения по весу в функцию вводится штраф за превышение вместимости. Сложность метода зависит от числа итераций K и для каждого шага составляет $O(n)$ (подсчёт веса и стоимости), итоговая сложность $O(Kn)$. Метод не гарантирует оптимального решения, но на практике часто находит близкое к нему за удовлетворительное время.

В таблице 1 приведено сравнение асимптотической сложности и свойств рассмотренных алгоритмов.

Таблица 1

Сравнение алгоритмов решения задачи о рюкзаке 0/1

Алгоритм	Сложность	Гарантия оптимальности	Применимость
Полный перебор	$O(2^n)$	Да	$n \leq 20$
Жадный	$O(n \log n)$	Нет	Любые n
Динамическое программирование	$O(nW)$	Да	$n \leq 10^4, W \leq 10^6$
Имитация отжига	$O(Kn)$	Нет	Большие n и W

Анализ литературных источников показал, что для практических задач часто используют гибридные подходы, комбинирующие точные методы для небольших подзадач и метаэвристики для общей структуры. В ходе данной практики были реализованы все четыре алгоритма в чистом виде с целью сравнения их производительности и точности на различных размерах входных данных, а также демонстрации неоптимальности жадной стратегии на специальных примерах.

Практическая реализация

Программный комплекс реализован на языке Python 3.10. В работе задействованы следующие библиотеки: стандартные модули random, time, typing, csv, os; для обработки данных и визуализации – pandas, matplotlib; для веб-сервера – FastAPI и uvicorn. Для модульного тестирования использовался модуль pytest. Веб-интерфейс включает серверную часть на FastAPI и клиентское одностраничное приложение на React.

Архитектура приложения

data_generator.py – функции генерации случайных предметов generate_random_items() с настраиваемыми максимальными весом и стоимостью, а также три специальные функции generate_tricky_case_1/2/3, возвращающие провокационные наборы с известной неоптимальностью жадного алгоритма.

algorithms.py – реализации всех четырёх алгоритмов:

- brute_force(items, capacity) – рекурсивный полный перебор, возвращает максимальную стоимость и список индексов выбранных предметов.
- greedy(items, capacity) – сортирует предметы по удельной стоимости и последовательно добавляет, пока хватает вместимости.
- dynamic_programming(items, capacity) – классическое ДП с восстановлением ответа путём обратного прохода по таблице.
- simulated_annealing(items, capacity, initial_temp=1000.0, cooling_rate=0.95, steps_per_temp=100, min_temp=0.1, penalty_factor=0.5) – имитация отжига с битовым представлением решения. Функция стоимости вычисляет суммарную стоимость предметов; при превышении вместимости к стоимости добавляется штраф, пропорциональный превышению. Соседнее состояние генерируется случайным переключением одного бита. Параметры подбирались экспериментально и обеспечивают компромисс между качеством и скоростью сходимости.

benchmark.py – скрипт автоматизированного тестирования. Для ряда n (5, 10, 15, 20, 25, 30, 40, 50, 75, 100) генерируется по 10 случайных наборов. Вместимость рюкзака устанавливается равной половине суммарного веса предметов (что создает интересные с точки зрения оптимизации ситуации). Для $n \leq 20$ эталоном служит результат полного перебора, для больших n – результат ДП. Замеряется время выполнения, вычисляется точность (отношение полученной стоимости к эталонной, в процентах). Результаты сохраняются в CSV-файл.

visualize.py – загружает CSV с результатами бенчмарка и строит три графика: зависимости среднего времени от n (логарифмическая шкала), зависимости точности от n для жадного и имитации отжига, а также числа итераций отжига от n . Графики сохраняются в папку plots в формате PNG.

app.py – веб-приложение на FastAPI. Предоставляет эндпоинты:

- POST /run – запуск всех алгоритмов на сгенерированных (или провокационных) данных и возврат результатов в JSON;
- GET /plots – список доступных графиков; GET /plot/{filename} — отдача файла графика;
- GET / – отдача клиентской HTML-страницы. Статические файлы раздаются через StaticFiles.

static/index.html – интерфейс пользователя с полями ввода n и вместимости, чекбоксом выбора провокационного набора, кнопкой запуска и зонами для отображения таблицы результатов и графиков. Взаимодействие с сервером осуществляется посредством асинхронных HTTP-запросов (fetch).

Взаимосвязь модулей: `data_generator` и `algorithms` являются библиотечными и используются как в скрипте бенчмарка, так и в веб-сервере. Визуализация выполняется отдельным скриптом по накопленным данным. Такая модульная архитектура позволяет независимо расширять и тестировать компоненты.

Детали реализации алгоритмов

Полный перебор выполняется рекурсивно: на каждом шаге для текущего предмета либо включаем его (если позволяет вес), либо пропускаем. Сложность $O(2^n)$ ограничивает применение $n \leq 20$.

Жадный алгоритм сортирует список кортежей (вес, стоимость, исходный индекс) по убыванию отношения v/w . Затем проходит по отсортированному списку и добавляет предмет, если его вес не превышает оставшуюся вместимость. Возвращает стоимость и индексы выбранных предметов.

Динамическое программирование создает двумерный список dp размера $(n + 1) \times (W + 1)$, заполнение производится двойным циклом. Восстановление набора предметов выполняется обратным ходом: начиная с ячейки $dp[n][W]$, если значение изменилось по сравнению с предыдущей строкой, предмет включается. Используется встроенный в Python тип `list`, что для больших W может быть затратно по памяти, но в ходе экспериментов вместимость не превышала нескольких тысяч, поэтому проблем не возникало.

Имитация отжига оперирует битовой строкой `solution` длины n , инициализируемой случайным образом. На каждой итерации случайно инвертируется один бит. Для нового решения вычисляется вес и стоимость; при превышении вместимости на $over = weight - capacity$ применяется штраф $penalty = over \times penalty_factor$, и эффективная стоимость становится $value - penalty$. Если новое решение лучше (эффективная стоимость выше), переход выполняется безусловно. Если хуже, переход происходит с вероятностью $e^{-\Delta/T}$, где T – текущая температура. Параметры по умолчанию: начальная температура 1000, коэффициент охлаждения 0.95, 100 итераций на одном температурном уровне, минимальная температура 0.1. Внешний цикл продолжается, пока $T > 0.1$. Такой набор обеспечивает достаточное количество итераций (порядка 200-300) и

на тестовых задачах показывает устойчивую сходимость к решениям, близким к оптимальным.

Тестирование и апробация

Методика тестирования

Для объективного сравнения алгоритмов разработан скрипт `benchmark.py`, который последовательно перебирает значения n из списка [5, 10, 15, 20, 25, 30, 40, 50, 75, 100]. Для каждого n генерируется 10 случайных наборов предметов с максимальными весом и стоимостью, адаптированными под размер задачи (до 100 для $n \leq 30$, до 200 для больших n). Вместимость рюкзака устанавливается равной $0.5 \times \sum w_i$, что не позволяет взять все предметы и делает задачу нетривиальной. В качестве эталонного решения для $n \leq 20$ используется результат полного перебора, для $n > 20$ – результат динамического программирования (оба гарантируют оптимальность). Для каждого алгоритма фиксируются время выполнения (по показаниям `time.perf_counter()`) и полученная стоимость. Точность в процентах вычисляется как $(value / true_value) \times 100\%$. Для имитации отжига дополнительно сохраняется число выполненных итераций.

Кроме случайных данных, проведено тестирование на трёх специальных провокационных наборах, демонстрирующих неоптимальность жадного алгоритма. Эти наборы вручную подобраны так, чтобы предмет с высокой удельной стоимостью занимал почти весь рюкзак, в то время как комбинация других предметов давала большую общую стоимость.

Все результаты сохраняются в CSV-файлы для последующего анализа и визуализации.

Результаты экспериментов на провокационных наборах данных

Рассмотрим один из провокационных наборов (набор №1): три предмета с весами и стоимостями (6,30), (5,25), (5,24) и вместимость $W = 10$. Оптимальное решение – взять второй и третий предметы (суммарный вес 10, стоимость 49). Жадный алгоритм, сортируя по удельной стоимости, первым берёт предмет (6,30) и не может добавить ни один из оставшихся – итоговая стоимость 30. Аналогичную ошибку жадный алгоритм допускает на провокационных примерах 2 и 3. Результаты работы всех алгоритмов на данных наборах приведены в таблице 2.

Таблица 2

Результаты работы алгоритмов на провокационных наборах данных

Алгоритм	Итоговая стоимость	Индексы выбранных элементов	Время, с
brute_force	49	[1, 2]	8.7079999574956e-06
greedy	30	[0]	3.417000016270322e-06
DP	49	[1, 2]	7.291999963854323e-06
simulated_annealing	49	[1, 2]	0.15155062500002714
brute_force	180	[1, 2]	6.708000000799075e-06
greedy	100	[0]	3.917000071851362e-06
DP	180	[1, 2]	3.545899994605861e-05
simulated_annealing	180	[1, 2]	0.15181858300002204
brute_force	98	[1, 2]	6.916999950590252e-06
greedy	95	[0]	2.79200003205915e-06
DP	98	[1, 2]	1.016700002764992e-05
simulated_annealing	98	[1, 2]	0.1518344999999499

Результаты эксперимента на случайных наборах данных

Усредненные результаты по 10 испытаниям для ряда n представлены csv файле benchmark_results.csv. Здесь не приводятся, т.к. занимают много места. Полный перебор применялся только до $n = 20$, далее отключен из-за неприемлемого времени.

Анализ результатов показывает:

Полный перебор даже при $n = 20$ демонстрирует экспоненциальный рост времени (единицы миллисекунд при $n = 10$ и уже единицы секунд при $n = 20$), полностью подтверждая теоретическую оценку.

Жадный алгоритм работает стабильно быстро для всех n (менее 0.0003 с), но его точность не всегда идеальна. Это объясняется тем, что при большем выборе вероятность неоптимального жадного решения возрастает.

Динамическое программирование обеспечивает 100% точность, однако время выполнения существенно растёт с увеличением вместимости и n . При $n = 100$ $W \approx 2500$ среднее время составило около 0.065 с, что всё ещё приемлемо для многих приложений.

Имитация отжига демонстрирует хороший баланс: точность выше 98%, несмотря на большее время, чем у ДП и жадного алгоритма. Количество итераций медленно растёт с увеличением n , так как алгоритм выполняет фиксированное число температурных шагов (определяемое параметрами охлаждения) и на каждом шаге обрабатывает все n битов. Рост числа итераций связан с небольшим изменением момента достижения минимальной температуры из-за стохастической природы. Время работы имитации отжига для $n = 100$ составило около 28.9 с, что дольше ДП и жадного алгоритма при сохранении высокой точности. Для больших n и особенно больших W , где ДП становится нереализуемым из-за памяти, имитация отжига остаётся работоспособной альтернативой.

Визуализация

С помощью visualize.py построены графики, которые наглядно отражают полученные зависимости:

- График времени от n (ось Y логарифмическая) показывает, что кривая полного перебора быстро уходит вверх, жадный метод практически горизонтален, ДП демонстрирует полиномиальный рост, а имитация отжига растёт заметно медленнее ДП.
- График точности приближенных методов показывает, что SA устойчиво превосходит жадный, причем разрыв увеличивается с ростом n .
- График итераций отжига демонстрирует небольшой восходящий тренд, что подтверждает незначительный рост вычислительных затрат алгоритма.

Все графики сохранены в папке plots и доступны через веб-интерфейс.

Апробация веб-интерфейса

Веб-интерфейс успешно протестирован в локальной среде (запуск `uvicorn app:app`). Форма позволяет ввести n и вместимость, отправить запрос и получить таблицу с результатами всех алгоритмов, временем и количеством итераций отжига. При выборе опции «Провокационный набор» генерируется один из трех специальных примеров. Раздел с графиками загружает ранее построенные изображения, давая пользователю наглядное представление о производительности методов. Интерфейс обеспечивает удобную демонстрацию работы системы и может быть использован в учебном процессе.

Модульное тестирование компонентов

Для обеспечения корректности реализации каждого программного модуля разработан комплект автоматических модульных тестов с использованием фреймворка

pytest. Тесты охватывают ключевые функции генератора данных, алгоритмов решения задачи, скрипта бенчмаркинга и веб-приложения, проверяя как типичные сценарии использования, так и граничные условия.

Тесты алгоритмов (файл `test_algorithms.py`) включают:

- Полный перебор – проверка правильности вычисления оптимальной стоимости на простом наборе, обработка пустого списка предметов, случая, когда ни один предмет не влезает, и нулевой вместимости.
- Жадный алгоритм – проверка совпадения с оптимумом на наборе, где жадный выбор корректен, демонстрация неоптимальности на специально подобранном примере (сравнение с динамическим программированием), обработка пустого входа, а также тест на ситуацию с предметами нулевого веса (ожидается исключение `ZeroDivisionError` как индикатор необходимости защиты в коде).
- Динамическое программирование – проверка оптимальности на простом наборе, проверка согласованности восстановленного набора предметов с полным перебором (вес не превышает вместимость, суммарная стоимость совпадает с рассчитанной), корректность для большой вместимости.
- Имитация отжига – проверка детерминированности при фиксированном начальном значении генератора случайных чисел (`random_seed`), гарантия допустимости всех возвращаемых решений ($\text{вес} \leq \text{вместимость}$) при многократных запусках, оценка качества приближения на малом наборе (отклонение от оптимума не более 1), обработка пустого списка предметов.

Тесты генератора данных (`test_data_generator.py`) подтверждают, что функция `generate_random_items` создаёт список требуемой длины, а веса и стоимости лежат в заданных диапазонах. Для каждого из трёх провокационных наборов с помощью полного перебора проверено известное оптимальное значение, что гарантирует корректность этих примеров для демонстрации ошибок жадного алгоритма.

Тесты скрипта бенчмаркинга (`test_benchmark.py`) проверяют:

- Корректность вычисления точности (`accuracy_percent`) относительно эталонного решения, включая случай, когда оптимальная стоимость равна нулю.
- Обработку пустого списка размерностей `N_VALUES` – функция не должна аварийно завершаться, а создаёт CSV-файл только с заголовками.
- Структуру и содержимое выходного CSV-файла для провокационных наборов (ожидаемое число строк и столбцов).
- Фактическую запись замеров времени (проверка с подменой `time.perf_counter`) и логику установки минимальной вместимости не менее 1.

Тесты веб-приложения (`test_app.py`) используют `TestClient` из `FastAPI` и проверяют:

- Корректную отдачу HTML-страницы по корневому маршруту (статус 200, content-type text/html).
- Успешный ответ эндпоинта /run с корректным JSON, содержащим все ожидаемые алгоритмы (для $n \leq 20$ добавляется полный перебор) и обязательные поля (value, selected, time_seconds).
- Работу с флагом use_tricky и обработку некорректных входных данных (возврат ошибки 400 при $n = 0$).
- Эндпоинт /plots в случае отсутствия папки с графиками возвращает пустой список.
- Запрос несуществующего файла графика возвращает 404.

Все тесты успешно выполняются, подтверждая корректность реализации и устойчивость к крайним случаям. Модульное тестирование проводилось параллельно с разработкой, что позволило своевременно выявлять и устранять дефекты.

Заключение

В ходе учебной практики полностью выполнены запланированные работы. Достигнуты следующие основные результаты:

- Разработан модуль генерации тестовых данных, включающий случайные и три специальных провокационных набора, на которых наглядно проявляется неоптимальность жадного алгоритма.
- Реализованы четыре алгоритма решения задачи о рюкзаке 0/1: полный перебор, жадный, динамическое программирование и имитация отжига. Программный код написан с соблюдением принципов модульности и сопровождается подробными комментариями.
- Создан скрипт автоматизированного бенчмаркинга, позволяющий проводить многократное тестирование на случайных данных, измерять время и точность, а также сохранять результаты в CSV.
- Реализован модуль визуализации, строящий графики ключевых зависимостей и сохраняющий их в графическом формате.
- Разработан веб-интерфейс на базе FastAPI, предоставляющий REST API для запуска алгоритмов и получения графиков, а также клиентская страница для удобного взаимодействия с системой.
- Проведён вычислительный эксперимент на 10 размерах задачи (от 5 до 100 предметов) с 10-кратной повторностью, подтвердивший теоретические оценки сложности и точности методов. Экспериментально установлено, что:
 - полный перебор применим только до $n \approx 20$;
 - динамическое программирование гарантирует точное решение, но его время существенно зависит от вместимости и достигает нескольких секунд при $n = 100$;

- жадный алгоритм работает практически мгновенно, но его точность падает до 82% при больших n ;
- имитация отжига демонстрирует точность около 94–99% и время выполнения, значительно меньшее, чем у ДП, для задач большой размерности, что делает его перспективным для практических приложений.
- Продемонстрирована работоспособность системы в целом, включая веб-интерфейс и визуализацию.

Практическая значимость работы заключается в создании готового инструмента для сравнительного анализа алгоритмов задачи о рюкзаке, который может применяться в учебных целях, а также как прототип модуля принятия решений в логистических и ресурсных задачах. Перспективы развития включают добавление других метаэвристик (генетический алгоритм, муравьиный алгоритм), переход клиентской части на React, а также реализацию сохранения истории запусков и более гибких настроек параметров имитации отжига.

Список источников

1. Kirkpatrick S., Gelatt C. D., Vecchi M. P. Optimization by Simulated Annealing // Science. – 1983. – Vol. 220, No. 4598. – P. 671–680.
2. Martello S., Toth P. Knapsack Problems: Algorithms and Computer Implementations. – Chichester: Wiley, 1990. – 306 p.
3. FastAPI Documentation. – URL: <https://fastapi.tiangolo.com/> (дата обращения: 14.06.2026).
4. Python 3.10 Documentation. – URL: <https://docs.python.org/3.10/> (дата обращения: 10.06.2026).
5. Введение в оптимизацию: Имитация отжига. – URL: <https://habr.com/ru/articles/209610/> (дата обращения: 08.06.2026).
6. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. – 3-е изд. – М.: Вильямс, 2013. – 1328 с.